Markdown: Eine Projekt-README erstellen

Lernmaterial: CC BY 4.0 (siehe `LICENSE-CONTENT.txt`).

Ziel

Du kannst eine README-Datei mit Markdown strukturieren und so anpassen, dass andere den Zweck eines Projekts, seine Voraussetzungen und den Einstieg verstehen.

Was ist Markdown?

Markdown ist eine einfache Auszeichnungssprache für formatierten Text. Du schreibst Überschriften, Listen, Links und Codeblöcke mit gewöhnlichen Textzeichen. GitHub zeigt eine Markdown-Datei wie `README.md` als formatierte Seite an.

Eine README ist eine Einstiegsseite zu einem Projekt. Sie soll Fragen beantworten, die neue Leserinnen und Leser wahrscheinlich haben: Worum geht es? Was wird benötigt? Wie kann ich beginnen? Welche Dateien oder Bereiche sind wichtig?

Die Vorlage öffnen

Datei prüfen: `README-Projektvorlage.md` im Kurs-Repository.

Die Vorlage ist ein Beispiel für ein fiktives Projekt. Die Angaben darin sind Beispieldaten und müssen durch Informationen zum eigenen Projekt ersetzt werden. Übernimm keine erfundenen Voraussetzungen oder Befehle in eine echte Projektdokumentation.

Aufgabe: Lies die Vorlage und markiere Titel, Abschnitte, Listen und die Stelle, an der du später einen eigenen Link ergänzt.

Ausführung: Öffne die Datei im Kurs-Repository als Text oder auf GitHub. Vergleiche die Markdown-Schreibweise mit der formatierten Ansicht.

Kontrolle: Du findest einen Projekttitel, mehrere Abschnitte mit Überschriften, Listen und einen Abschnitt für weiterführende Informationen. Die Vorlage enthält absichtlich noch keinen Projektlink, weil die Ziel-Datei im eigenen Lern-Repository liegen muss.

Markdown-Bausteine

Überschriften

Eine oder mehrere Rauten am Zeilenanfang kennzeichnen eine Überschrift. Eine einzelne Raute ist die oberste Ebene; mehr Rauten ergeben untergeordnete Ebenen. Beginne mit einer Hauptüberschrift und ordne Unterabschnitte sinnvoll darunter ein.

```
# Projektname

## Zweck

### Einzelheit
```

Lasse eine Leerzeile zwischen Abschnitten. Wähle Überschriften nach ihrer Bedeutung und nicht nur nach dem gewünschten Schriftbild.

Absätze und Listen

Ein normaler Textblock wird als Absatz dargestellt. Eine Leerzeile trennt Absätze. Für ungeordnete Listen setzt du einen Bindestrich und ein Leerzeichen vor jeden Punkt:

```
- Ein erster Punkt
- Ein weiterer Punkt
```

Für eine nummerierte Schrittfolge verwendest du Zahlen mit Punkt:

```
1. Voraussetzung prüfen
2. Projektdateien öffnen
3. Ergebnis kontrollieren
```

Links

Ein Link besteht aus einem sichtbaren Linktext in eckigen Klammern und der Zieladresse in runden Klammern:

```
[Projektübersicht](README.md)
```

Verwende einen beschreibenden Linktext, damit auch ohne das Ziel zu öffnen klar ist, wohin der Link führt. Prüfe bei relativen Links, dass die Zieldatei im Projekt tatsächlich vorhanden ist.

Code und Codeblöcke

Kurze Dateinamen oder Befehle im Fließtext setzt du zwischen einzelne Backticks, zum Beispiel `README.md`. Mehrzeilige Befehle oder Quellcode setzt du zwischen drei Backticks. Eine Sprachangabe direkt nach den öffnenden Backticks kann die Darstellung einfärben:

````
```bash
printf '%s\n' "Hallo"
```
````

Dokumentiere nur Befehle, deren Wirkung du geprüft hast. Erkläre, wo sie ausgeführt werden und was sie bewirken. Weise auf mögliche Folgen hin, wenn sie Dateien verändern oder löschen.

Tabellen und Sonderzeichen

Markdown-Tabellen verwenden Pipes (`|`) als Spaltentrenner. Ein Pipe-Zeichen, das als normaler Text in einer Tabellenzelle steht, muss je nach Darstellung mit einem Backslash maskiert werden. Das ist fehleranfällig und kann zwischen Editoren unterschiedlich aussehen.

Verwende für eine kurze Gegenüberstellung lieber eine Liste mit Begriff und Erklärung:

- **Pipe (`|`):** leitet die Ausgabe eines Befehls an den nächsten weiter.
- **Umleitung (`>`):** schreibt die Ausgabe in eine Datei und kann vorhandenen Inhalt überschreiben.

Tabellen sind sinnvoll, wenn viele Einträge gleichartige Spalten haben und keine Zelle selbst Pipes enthält. Bei README-Dateien sind Überschriften und Listen oft einfacher zu lesen und zu bearbeiten.

Eine Projekt-README planen

Eine README muss nicht jede Einzelheit eines Projekts erklären. Wähle Abschnitte, die zur Zielgruppe und zum Projekt passen. Mögliche Bestandteile sind:

- **Projektname und Kurzbeschreibung:** Was ist das Projekt und welchen Zweck hat es?
- **Voraussetzungen:** Welche Programme, Konten oder Daten werden tatsächlich benötigt?
- **Einstieg oder Installation:** Welche geprüften Schritte führen zum ersten Ergebnis?
- **Nutzung:** Wie wird das Projekt verwendet? Zeige nur Befehle, die es wirklich gibt.
- **Dateien und Ordner:** Wo liegen zentrale Dateien?
- **Prüfen und Fehlerhilfe:** Woran erkennt man ein korrektes Ergebnis und wo beginnt die Fehlersuche?
- **Lizenz oder Mitwirkung:** Nur aufführen, wenn die Angaben für das Projekt bekannt und zutreffend sind.

Nicht jedes Projekt braucht alle Abschnitte. Lass Unbekanntes weg oder kennzeichne es ausdrücklich als noch festzulegen, statt Fakten zu erfinden.

Übung: README für dein Lern-Repository

Lernziel: Du erstellst eine README, die dein Lern-Repository verständlich vorstellt und den Einstieg in die Kursdateien erleichtert.

Voraussetzung: Richte zuerst dein Lern-Repository nach Teil 1 ein und lege dort `.github/workflows/01-hallo-linux.yml` an. So enthält dein Repository eine vorhandene Datei, auf die du in der README verlinken kannst.

Datei anlegen oder ändern: `README.md` im eigenen Lern-Repository. Ändere nicht die README des Kurs-Repositorys für diese Übung.

Aufgabe

Erstelle eine README mit:

1. einem aussagekräftigen Titel,
2. einem kurzen Absatz zum Zweck deines Lern-Repositorys,
3. einer Liste mit mindestens drei Bereichen oder Dateien, die im Kurs vorkommen,
4. einem Abschnitt zu Voraussetzungen, die für dein Repository tatsächlich gelten,
5. einem Abschnitt „Erste Schritte“ mit den nächsten Kursaktionen,

6. mindestens einem funktionierenden Link zu einer Datei oder einem Ordner, der in deinem Repository vorhanden ist.

Nutze die Vorlage `README-Projektvorlage.md` als Orientierung. Passe sie an und entferne alle Beispieldaten, die nicht auf dein Projekt zutreffen. Schreibe keine erfundenen Installationsbefehle oder Testergebnisse hinein.

Ausführung

Öffne dein Lern-Repository auf GitHub und bearbeite dessen `README.md` im Browser. Schreibe die Inhalte mit Markdown-Syntax. Nutze **Preview**, um die formatierte Darstellung vor dem Speichern anzusehen. Da du Git-Grundlagen bereits gelesen hast, wähle den dort angelegten Arbeitsbranch, falls vorhanden. Andernfalls kannst du diese erste README-Änderung auf dem Standardbranch speichern.

Gib eine kurze Commit-Nachricht ein, die die Änderung beschreibt, und bestätige **Commit changes**. Falls GitHub anbietet, die Änderung auf einem neuen Branch zu speichern, kannst du diese Möglichkeit wählen; prüfe dabei, dass die Branch-Auswahl und der Commit-Schritt angezeigt werden. Der Ablauf ist derselbe wie im Abschnitt „Änderungen und Commits“ in Git-Grundlagen.

Ergebniskontrolle

Prüfe nach dem Speichern:

- Der Titel wird als Hauptüberschrift dargestellt.
- Abschnitte erscheinen in der richtigen Reihenfolge und Hierarchie.
- Listenpunkte sind getrennt und lesbar.
- Der Link führt zu einer vorhandenen Datei oder einem vorhandenen Ordner.
- Die README enthält keine unzutreffenden Beispieldaten oder ungeprüften Befehle.
- Auf der Startseite deines Repositorys zeigt GitHub die formatierte README an.

Typische Fehler

- **Markdown erscheint als Rohtext:** Prüfe, ob die Datei `README.md` heißt und die Markdown-Zeichen richtig gesetzt sind.
- **Ein Link führt ins Leere:** Vergleiche den Linkpfad mit dem tatsächlichen Dateinamen und der Ordnerstruktur.

- **Ein Codeblock bleibt offen:** Prüfe, ob auf den öffnenden drei Backticks ein passender schließender Backtick-Block folgt.
- **Eine Tabelle ist schwer zu lesen:** Ersetze sie durch Überschriften und Listen, besonders wenn Inhalte selbst Pipe-Zeichen enthalten.
- **Die README verspricht etwas, das nicht funktioniert:** Entferne oder korrigiere nicht überprüfte Schritte, Links und Aussagen.

Weiterlernen

GitHub rendert Markdown-Dateien mit GitHub Flavored Markdown, das Markdown um weitere Funktionen ergänzt. Die GitHub-Dokumentation zum Schreiben auf GitHub[1] erklärt die unterstützten Formatierungen. Grundlegende Markdown-Syntax beschreibt die Markdown-Referenz[2].

[1] https://docs.github.com/en/get-started/writing-on-github
[2] https://www.markdownguide.org/basic-syntax/